Transacciones, niveles de aislamiento y propiedades ACID

1. Definición de transacción

Una transacción de base de datos es una serie de una o más operaciones ejecutadas como una única unidad atómica de trabajo. Esto significa que, o bien todas las operaciones de la transacción se completan con éxito, o bien ninguna de ellas se aplica a la base de datos.

Las transacciones se utilizan para garantizar la coherencia e integridad de los datos, asegurando que la base de datos siga siendo coherente incluso en caso de fallos o errores del sistema. La característica clave de las transacciones es que son **atómicas, consistentes, aisladas y duraderas** (ACID), las cuatro propiedades que garantizan la fiabilidad de la base de datos.

2. Niveles de aislamiento en bases de datos relacionales

La mayoría de los DBMS ofrecen varios niveles de aislamiento de transacciones, que controlan el grado de bloqueo que se produce al seleccionar datos. En muchas aplicaciones, la mayoría de las transacciones se pueden construir de forma que no requieran niveles altos de aislamiento (por ejemplo, `SERIALIZABLE`), lo que reduce la sobrecarga de bloqueos en el sistema.

El programador debe analizar cuidadosamente el código de acceso a la base de datos para garantizar que relajar el aislamiento no provoque errores difíciles de encontrar. Por el contrario, si se utilizan niveles de aislamiento más altos, aumenta la posibilidad de interbloqueos (*deadlocks*), lo que también requiere técnicas cuidadosas de análisis y programación para evitarlos.

A continuación se describen los niveles, del más alto al más bajo.

2.1. Serializable

Es el nivel de aislamiento más alto.

En un DBMS con control de concurrencia basado en bloqueos, la serialización requiere que los bloqueos de lectura y escritura (adquiridos sobre los datos seleccionados) se liberen hasta el final de la transacción. También se deben adquirir **bloqueos de rango** cuando una consulta `SELECT` usa una cláusula `WHERE` con un rango, especialmente para evitar el fenómeno de las **lecturas fantasma**.

Cuando se utiliza control de concurrencia no basado en bloqueos, no se adquieren bloqueos; sin embargo, si el sistema detecta una colisión de escritura entre varias transacciones con-

currentes, solo una de ellas puede confirmarse (*commit*). Este enfoque se relaciona con el **aislamiento de instantáneas** (*snapshot isolation*).

La norma ISO/IEC 9075:1992 (lenguaje de base de datos SQL) establece lo siguiente:

> «Se garantiza que la ejecución de transacciones SQL simultáneas en el nivel de aislamiento SERIALIZABLE sea serializable. Una ejecución serializable se define como una ejecución de las operaciones de transacciones SQL que se ejecutan simultáneamente y que produce el mismo efecto que alguna ejecución en serie de esas mismas transacciones SQL. Una ejecución en serie es aquella en la que cada transacción SQL se ejecuta hasta su finalización antes de que comience la siguiente.»

2.2. Lectura repetible (*Repeatable Read*)

En este nivel, un DBMS con control de concurrencia basado en bloqueos mantiene los bloqueos de lectura y escritura (adquiridos sobre los datos seleccionados) hasta el final de la transacción. Sin embargo, los bloqueos de rango no se administran, por lo que **pueden ocurrir lecturas fantasma**.

En algunos sistemas también es posible el **sesgo de escritura** (*write skew*). Este fenómeno ocurre cuando dos transacciones concurrentes leen los mismos datos y, con base en lo que leyeron, cada una actualiza datos distintos. Ninguna sobrescribe a la otra, pero la combinación de ambas escrituras deja la base de datos en un estado que no se habría producido si se hubieran ejecutado una tras otra (por ejemplo, violando una regla de negocio que ambas verificaron por separado).

2.3. Lectura confirmada (*Read Committed*)

En este nivel, un DBMS con control de concurrencia basado en bloqueos mantiene los bloqueos de escritura hasta el final de la transacción, pero los bloqueos de lectura se liberan en cuanto termina la operación `SELECT`, por lo que **pueden ocurrir lecturas no repetibles**. Al igual que en el nivel anterior, los bloqueos de rango no se gestionan.

En pocas palabras, la lectura confirmada garantiza que cualquier dato leído ya esté confirmado en el momento en que se lee. Solo impide que el lector vea datos intermedios, no confirmados o «sucios». No promete que, si la transacción vuelve a realizar la misma lectura, encontrará los mismos datos: estos pueden cambiar después de haberse leído.

2.4. Lectura no confirmada (*Read Uncommitted*)

Es el nivel de aislamiento más bajo. En este nivel se permiten las **lecturas sucias**, por lo que una transacción puede ver cambios todavía no confirmados realizados por otras transacciones.

2.5. Nivel de aislamiento predeterminado

El nivel de aislamiento predeterminado varía bastante entre DBMS. La mayoría de las bases de datos con soporte de transacciones permiten al usuario establecer cualquier nivel. Algunos DBMS también requieren sintaxis adicional en una sentencia `SELECT` para adquirir bloqueos (por ejemplo, `SELECT ... FOR UPDATE` para adquirir bloqueos de escritura exclusivos sobre las filas consultadas).

3. Propiedades ACID

En el contexto de las bases de datos relacionales, las propiedades ACID son las características fundamentales que deben poseer los sistemas de gestión de bases de datos (DBMS) para garantizar la confiabilidad y solidez de las transacciones. ACID es un acrónimo de **atomicidad, consistencia, aislamiento y durabilidad**. Cuando se mantienen estas propiedades, la base de datos ofrece garantías sólidas sobre la integridad y corrección de los datos, incluso en entornos complejos, multiusuario o de alta carga.

3.1. Atomicidad

Garantiza que una transacción se complete por completo o no se ejecute en absoluto. Todas las operaciones de una transacción se tratan como una unidad única e indivisible. Si falla alguna operación, toda la transacción se considera fallida y todos los cambios realizados durante ella se revierten (*rollback*). La atomicidad evita que la base de datos quede en un estado inconsistente por transacciones parcialmente completadas.

Ejemplo: en un sistema bancario, una transferencia entre dos cuentas implica un débito en una cuenta y un crédito en otra. Si una de estas operaciones falla, la atomicidad garantiza que la otra también se revierta, de modo que nunca se descuente dinero de una cuenta sin abonarse en la otra.

3.2. Consistencia

Garantiza que la base de datos permanezca en un estado consistente antes y después de una transacción. La base de datos debe comenzar en un estado consistente, se deben respetar todas las restricciones y reglas de negocio, y al terminar la transacción debe volver a quedar en un estado consistente. Esto incluye las restricciones del esquema, como llaves únicas, llaves foráneas y restricciones de verificación (`CHECK`).

Ejemplo: en un sistema de inventario, una regla puede indicar que las existencias de un producto no pueden ser negativas. Si una transacción intenta reducirlas por debajo de cero, se cancela, preservando la consistencia de los datos.

3.3. Aislamiento

Garantiza que cada transacción se ejecute de forma aislada respecto a otras transacciones concurrentes: las operaciones de una transacción no son visibles para las demás hasta que se completa. El aislamiento evita efectos indeseables cuando varias transacciones acceden o modifican los mismos datos simultáneamente, como actualizaciones perdidas, lecturas sucias o lecturas no repetibles.

Existen distintos grados de aislamiento: desde niveles bajos, que priorizan el rendimiento y la concurrencia pero pueden permitir inconsistencias, hasta niveles altos, que priorizan la coherencia de los datos a costa del rendimiento. Lograr el equilibrio adecuado depende de los requisitos de cada aplicación.

3.4. Durabilidad

Garantiza que, una vez que una transacción se confirma con éxito, sus efectos son permanentes y no se perderán, incluso si ocurre una falla del sistema. La durabilidad se logra mediante técnicas como el **registro de escritura anticipada** (*Write-Ahead Logging*, WAL) y el registro en diario (*journaling*), donde los cambios se escriben primero en un registro antes de aplicarse a la base de datos. Ante una falla, el registro se usa para recuperar los datos y restaurar la base de datos a un estado consistente. Esta propiedad es especialmente importante en aplicaciones con datos críticos, como las transacciones financieras.

Referencias

- AcademiaLab. (s. f.). *Aislamiento (sistemas de bases de datos)*. https://academia-lab.com/enciclopedia/aislamiento-sistemas-de-bases-de-datos/
- AppMaster. (2026, 29 de julio). *¿Qué es una transacción de base de datos?* https://appmaster.io/es/blog/que-es-una-transaccion-de-base-de-datos
- Team, A. (2020, 21 de marzo). *Propiedades ÁCIDAS (Atomicidad, consistencia, aislamiento, durabilidad)*. AppMaster. https://appmaster.io/es/glossary/propiedades-acidas-atomicidad-consistencia-aislamiento-durabilidad